

Title Page

Course: CO2 - Algorithm Analysis and Design

Assessment: AT2

Question No: 7

Title: Brute- Force Approach to Find the Farthest Pair of Delivery Points

Name: ____ARCHANA DEVI.P_____

Register Number: ____192521192_____

Department: Information Technology

Date: ____29.07.2026_____

1. Aim

To identify the farthest pair of delivery points using the brute- force approach by calculating the Euclidean distance between every possible pair of coordinates.

2. Problem Statement

A navigation system processes the coordinates of multiple delivery points to determine the farthest pair of locations for route planning. The brute- force approach computes the distance between every possible pair and identifies the pair with the maximum distance.

3. Algorithm

1. Read the number of delivery points.
2. Input the coordinates of all points.
3. Initialize the maximum distance to zero.
4. Compare every point with every other point.
5. Compute the Euclidean distance between each pair.
6. Update the maximum distance whenever a larger distance is found.
7. Display the farthest pair of points and the maximum distance.

4. Source code



JS

TS

main.c



Share

Run

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main()
5  {
6      int n;
7
8      printf("Enter number of delivery points: ");
9      scanf("%d", &n);
10
11     float x[n], y[n];
12
13     for(int i=0; i<n; i++)
14     {
15         printf("Enter coordinates of Point %d (x y): ", i+1);
16         scanf("%f %f", &x[i], &y[i]);
17     }
18
19     float maxDistance = 0;
20     int p1 = 0, p2 = 1;
21
22     for(int i=0; i<n; i++)
```

main.c

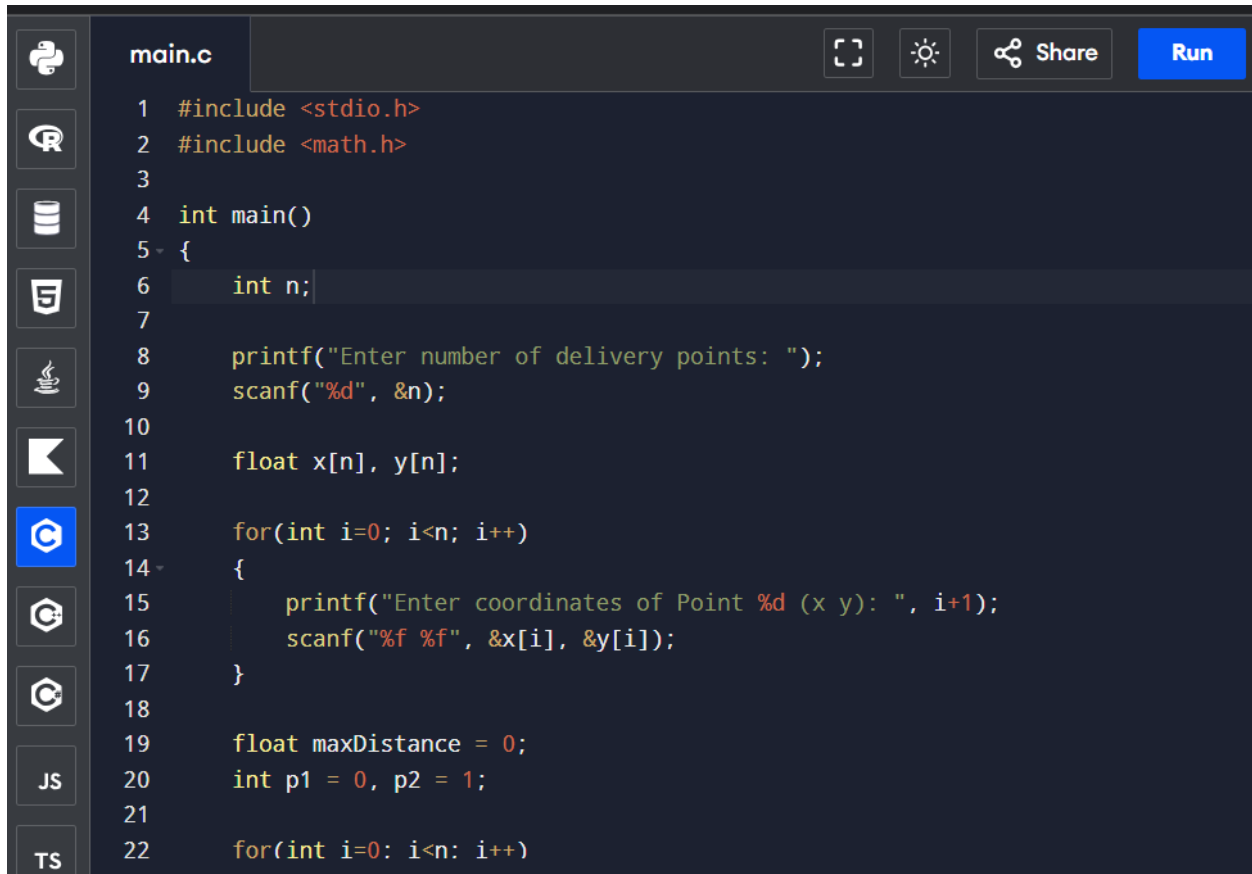


Share

Run

```
21
22     for(int i=0; i<n; i++)
23     {
24         for(int j=i+1; j<n; j++)
25         {
26             float dx = x[i] - x[j];
27             float dy = y[i] - y[j];
28
29             float distance = sqrt(dx*dx + dy*dy);
30
31             if(distance > maxDistance)
32             {
33                 maxDistance = distance;
34                 p1 = i;
35                 p2 = j;
36             }
37         }
38     }
39
40     printf("\nFarthest Pair:\n");
41     printf("Point %d (%.2f, %.2f)\n", p1+1, x[p1], y[p1]);
42     printf("Point %d (%.2f, %.2f)\n", p2+1, x[p2], y[p2]);
43     printf("Maximum Distance = %.2f\n", maxDistance);
44
45     return 0;
46 }
```

5. Execution Output



```
main.c
1 #include <stdio.h>
2 #include <math.h>
3
4 int main()
5 {
6     int n;
7
8     printf("Enter number of delivery points: ");
9     scanf("%d", &n);
10
11     float x[n], y[n];
12
13     for(int i=0; i<n; i++)
14     {
15         printf("Enter coordinates of Point %d (x y): ", i+1);
16         scanf("%f %f", &x[i], &y[i]);
17     }
18
19     float maxDistance = 0;
20     int p1 = 0, p2 = 1;
21
22     for(int i=0; i<n; i++)
```

6. Time Complexity Analysis

Let the number of delivery points be n .

The algorithm compares every possible pair of points.

Number of comparisons:

Each distance calculation requires constant time $O(1)$.

Therefore,

Time Complexity = $O(n^2)$

This quadratic complexity means the execution time increases rapidly as the number of delivery points increases.

7. Space Complexity Analysis

The algorithm stores the x and y coordinates of all points.

Storage required:

- x array $O(n)$
- y array $O(n)$
- Other variables $O(1)$

Overall,

Space Complexity = $O(n)$

8. Result

The brute- force algorithm successfully identifies the farthest pair of delivery points by computing the Euclidean distance between every possible pair of coordinates.

9. Conclusion

The brute- force method is simple to implement and guarantees the correct solution by checking every possible pair of points. However, since it has a time complexity of $O(n^2)$, it becomes inefficient for large datasets. More advanced algorithms are preferred for handling large numbers of delivery points in real- time navigation systems.